Procesos de Desarrollo de Software

CLASE 8 – CLEAN ARCHITECTURE

Agenda de la clase

- Patrones de software (Singleton + DAO)
- Clean architecture
  - Separación por capas
  - Qué es un Service
  - Qué es un Repository
  - Cómo se organiza un proyecto Spring Boot
  - Aplicarlo a MagicCafe

Patrones de Diseño

- "Soluciones conocidas a problemas conocidos"
- **"*Son descripciones de clases y objetos relacionados que están adaptados para resolver un problema de diseño general en un contexto determinado.*"**

  Erich Gamma, Richard Helm, John Vlissides y Ralph Johnson
- "***Consiste en un diagrama de objetos que forma una solución a un problema conocido y frecuente.***"

  Laurent Debrauwer – Patrones de diseño en Java

Partes esenciales

1. Nombre

Apoya la identificación e incorporación de ese concepto en nuestro conocimiento y memoria. Facilita la comunicación.



2. Problema que resuelve

Son problemas repetitivos, cuyas soluciones fueron probadas y fáciles de mantener.

3. Solución

Son una solución general a problemas conocidos pero se adaptan de acuerdo a contextos específicos.

Patrones de Diseño / Clasificación

- **Creacionales**: abstraen el proceso de creación/instanciación de los objetos. Se los suele utilizar cuando debemos crear objetos, complejos o no, tomando decisiones dinámicas en momento de ejecución.
- **Comportamiento**: resuelven cuestiones, complejas o no, de interacción entre objetos en momento de ejecución.
- **Estructurales**: resuelven cuestiones, generalmente complejas, de generación y/o utilización de estructuras complejas o que no están acopladas al dominio.

Clasificación

Creacionales	Comportamiento	Estructurales
• Factory Method • Simple Factory • Singleton • Abstract Factory • Builder • Prototype	• State • Strategy • Observer • Command • Template Method • Iterator • Memento • Visitor • Interpreter • Chain of Responsability • Mediator	• Adapter • Composite • Facade • Decorator • Proxy • Flyweight • Bridge

Arquitectura de Software

- La arquitectura de software representa la **estructura o estructuras** del sistema que consiste en **componentes** de **software**, las propiedades **externas** visibles de esos componentes y las **relaciones** entre ellos.
- Un **componente** es una pieza **de software** que puede ser código fuente, código binario, un ejecutable, o una biblioteca con una **interfaz definida**.

¿Qué es la interfaz de un Componente?

- Una **interfaz** establece las **operaciones externas** de un componente, las cuales determinan una **parte** del **comportamiento** del mismo.

¿Cómo se relacionan con otros componentes?

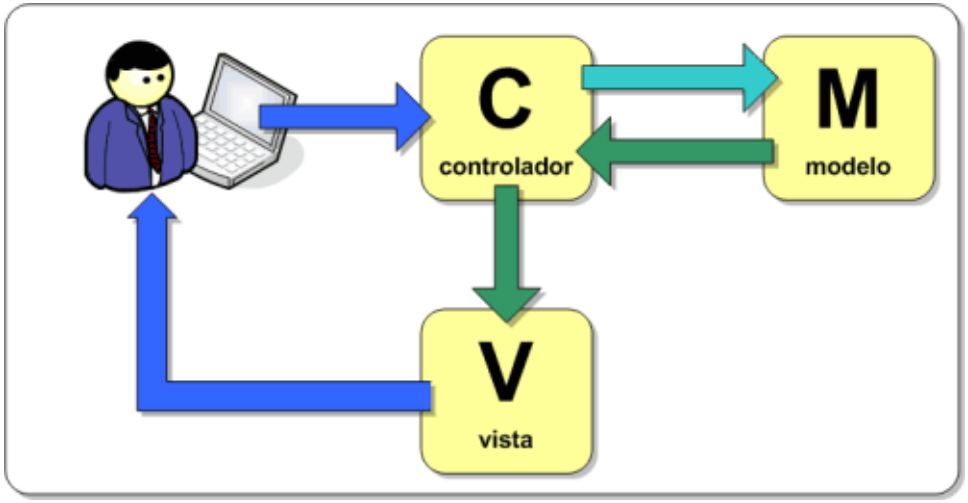
- Se representan las dependencias entre componentes o entre un componente y la interfaz de otro, es decir uno de ellos usa los servicios o funcionalidades del otro.

Estilos y patrones arquitectónicos

- Un **estilo arquitectónico** es una forma conceptual de cómo se creará/funcionará el sistema
- Un **patrón arquitectónico** describe una solución para implementar un estilo a nivel de subsistemas o módulos y sus relaciones. Es una forma de resolver un problema arquitectónico recurrente.
  - **MVC**, por ejemplo, resuelve el problema de separar la IU del modelo.

MVC

- Modelo-vista-controlador (***MVC***) es un ***patrón de arquitectura de software*** (también llamados *patrones arquitectónicos*) que ***separa*** la ***arquitectura*** en ***tres componentes***: el ***modelo,*** la ***vista*** y el ***controlador***.



Modelo

Está vinculado con la representación de los datos con los cuales el sistema opera. Está relacionado a la lógica y las reglas del negocio.

- Valida la especificación de la lógica detallada en el "**negocio**"
- Envía a la **Vista***, a través del controller*, los datos solicitados para que sean visualizados
- En el ***modelo*** se deberían realizar las validaciones explícitas del negocio.

Controlador

Responde a eventos (usualmente acciones del usuario) e invoca peticiones al modelo cuando se hace alguna solicitud sobre los datos.

- Es el ***intermediario*** entre el modelo y la vista
- Responde a eventos de los usuarios (físicos) y de otros sistemas
- Tiene la capacidad de traducir acciones del usuario a actualizaciones (o inserciones) del modelo
- Invoca peticiones al modelo
- En el ***controlador*** se debería volver a validar la consistencia de los datos y los campos requeridos. También se puede realizar una validación adicional contra un servicio externo o componente local.

Vista

Presenta los datos y su forma de interactuar en un formato adecuado para el usuario.

- Posee lógica para poder representar los datos de la forma más "*amigable*" para el usuario
- Envía acciones del usuario al controlador
- Solicita actualizaciones (o inserciones) al modelo **a través del controller**
- La comunicación entre la vista y el controlador se debería realizar mediante objetos de transferencia de datos (**DTO** - *Data Transfer Objects*)
- En la ***vista*** se puede validar campos requeridos y consistencia (números y/o letras donde corresponda, entre otras cosas).

Singleton

- Es un patrón creacional

¿Qué hace?

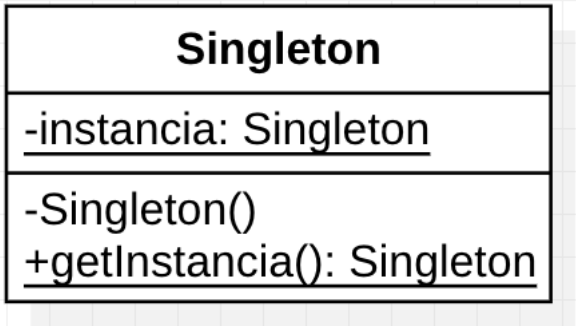
- Asegura que existirá una única instancia de una clase.

Se **sugiere** su utilización **cuando**:

- Se requiere tener una única instancia de una clase y que la misma sea accesible por terceros.

Singleton

- **Clase Singleton:** Clase que permitirá que sea instanciada una única vez a través de un método de clase. El constructor debe ser privado para que sea accesible, única y exclusivamente, por la misma clase.




```java
public class Singleton {
    private static Singleton instancia = null;

    public static Singleton getInstancia() {
        if (instancia == null) {
            instancia = new Singleton();
        }
        return instancia;
    }

    private Singleton() {
        // Constructor privado para evitar instanciación externa
    }
}
```

JDBC

- **J**ava **D**ata**B**ase **C**onnectivity
- Es una **API de Java** que permite a una aplicación **conectarse e interactuar con una base de datos**.
  - Conectarte a una base de datos
  - Ejecutar consultas SQL (SELECT)
  - Insertar datos (INSERT)
  - Actualizar (UPDATE)
  - Borrar (DELETE)
  - Leer resultados

Flujo Completo

1. Abrir conexión
2. Preparar query
3. Ejecutar
4. Procesar resultado
5. Cerrar conexión

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/magiccafe",
    "root",
    "password"
);
```

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM producto WHERE id = ?"
);
```

```
ResultSet rs = stmt.executeQuery();

while (rs.next()) {
    String nombre = rs.getString("nombre");
}
```

Ejemplo

```java
public void listarProductos() {
    try {
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/magiccafe",
            "root",
            "password"
        );

        String sql = "SELECT * FROM producto";
        PreparedStatement stmt = conn.prepareStatement(sql);

        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }

        conn.close();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Problemas de JDBC

- Mucho código repetido
- Difícil de mantener
- Mezcla lógica de negocio + SQL
- Alto acoplamiento
- Manejo manual de conexiones

Ejemplo

```java
public class PedidoService {

    public void crearPedido(Pedido pedido) {

        // 1. Lógica de negocio
        if (pedido.getItems().isEmpty()) {
            throw new RuntimeException("El pedido no puede estar vacío");
        }

        pedido.calcularTotal();
        pedido.setEstado("CREADO");

        // 2. Acceso a base de datos mezclado
        try {
            Connection conn = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/magiccafe",
                    "root",
                    "password"
            );

            String sql = "INSERT INTO pedidos (cliente, total, estado) VALUES (?, ?, ?)";

            PreparedStatement stmt = conn.prepareStatement(sql);
            stmt.setString(1, pedido.getCliente());
            stmt.setDouble(2, pedido.getTotal());
            stmt.setString(3, pedido.getEstado());

            stmt.executeUpdate();

            conn.close();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

- ¿Qué responsabilidades tiene?
  - Valida el pedido
  - Calcula el total
  - Cambia el estado
  - Se conecta a la base de datos
  - Arma SQL
  - Ejecuta SQL
  - Maneja errores de SQL

Mejorando el ejemplo

- Creamos una clase específica para guardar pedidos

```java
public class PedidoDAO {

    public void guardar(Pedido pedido) {
        try {
            Connection conn = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/magiccafe",
                    "root",
                    "password"
            );

            String sql = "INSERT INTO pedidos (cliente, total, estado) VALUES (?, ?, ?)";

            PreparedStatement stmt = conn.prepareStatement(sql);
            stmt.setString(1, pedido.getCliente());
            stmt.setDouble(2, pedido.getTotal());
            stmt.setString(3, pedido.getEstado());

            stmt.executeUpdate();

            conn.close();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```java
public class PedidoService {

    private PedidoDAO pedidoDAO = new PedidoDAO();

    public void crearPedido(Pedido pedido) {

        if (pedido.getItems().isEmpty()) {
            throw new RuntimeException("El pedido no puede estar vacío");
        }

        pedido.calcularTotal();
        pedido.setEstado("CREADO");

        pedidoDAO.guardar(pedido);
    }
}
```

DAO: Data Access Object

- Es un patrón de diseño cuya responsabilidad es encapsular el acceso a datos.
- Separa la lógica de negocio de la lógica de persistencia (DB, archivo, API externa).
- DAO **NO** tiene lógica de negocio.
  - ❌ Mal:

```
if (pedido.getTotal() > 1000)
```

"El Service no debería saber cómo se guarda un pedido. Solo debería saber que el pedido debe guardarse."

Ejemplo

```java
public interface PedidoDAO {

    void guardar(Pedido pedido);
    Pedido buscarPorId(int id);
    void actualizar(Pedido pedido);
    void eliminar(int id);
}
```

Continuando nuestro ejemplo…

```java
public class PedidoDAO {

    public void guardar(Pedido pedido) {

        try {
            Connection conn = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/magiccafe",
                    "root",
                    "password"
            );

            String sql = "INSERT INTO pedidos (cliente, total, estado) VALUES (?, ?, ?)";

            PreparedStatement stmt = conn.prepareStatement(sql);

            stmt.setString(1, pedido.getCliente());
            stmt.setDouble(2, pedido.getTotal());
            stmt.setString(3, pedido.getEstado());

            stmt.executeUpdate();

            conn.close();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

- El problema es que el DAO está creando conexiones manualmente.

Solucionando el problema

```java
public class DBConnection {

    private static DBConnection instance;
    private Connection connection;

    private DBConnection() {
        try {

            connection = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/magiccafe",
                    "root",
                    "password"
            );

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public static DBConnection getInstance() {
        if (instance == null) {
            instance = new DBConnection();
        }
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }
}
```

```java
public class PedidoDAO {

    public void guardar(Pedido pedido) {

        try {

            Connection conn = DBConnection
                    .getInstance()
                    .getConnection();

            String sql = "INSERT INTO pedidos (cliente, total, estado) VALUES (?, ?, ?)";

            PreparedStatement stmt = conn.prepareStatement(sql);

            stmt.setString(1, pedido.getCliente());
            stmt.setDouble(2, pedido.getTotal());
            stmt.setString(3, pedido.getEstado());

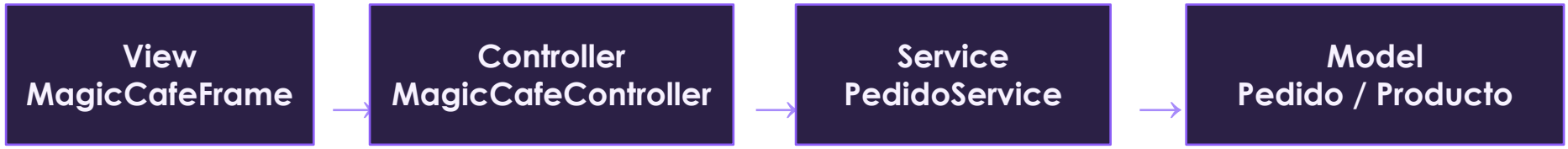
            stmt.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Magic Café - Práctica

- Evolucionando Swing Magic Cafe hacia persistencia con base de datos. Emplearemos:
  - MVC
  - DTO
  - JDBC
  - Singleton
  - DAO

Punto de partida



- El problema actual



Modelo de base de datos

- Tres tablas simples para explicar persistencia sin complejidad innecesaria.

productos
id
nombre
precio
emoji

pedidos
id
fecha
subtotal
descuento
total

pedido_items
id
pedido_id
producto_id
cantidad
subtotal

- 1 pedido tiene ‘N’ ítems y cada ítem tiene una referencia a un producto

Scripts para crear DB

```
CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  precio DECIMAL(10,2) NOT NULL,
  emoji VARCHAR(10) NOT NULL
);

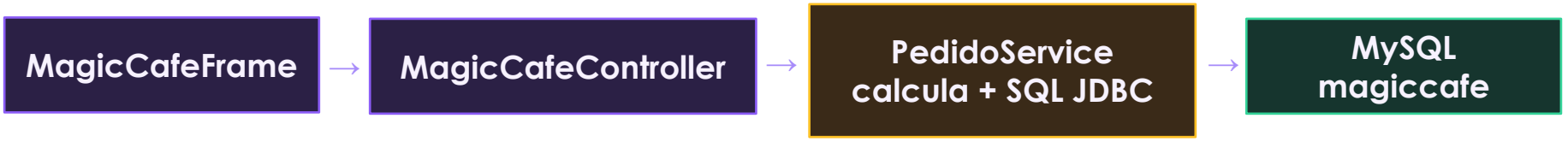
CREATE TABLE pedidos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  subtotal DECIMAL(10,2) NOT NULL,
  descuento VARCHAR(100) NOT NULL,
  total DECIMAL(10,2) NOT NULL
);
```

```
CREATE TABLE pedido_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  pedido_id INT NOT NULL,
  producto_id INT NOT NULL,
  producto_nombre VARCHAR(100) NOT NULL,
  cantidad INT NOT NULL,
  precio_unitario DECIMAL(10,2) NOT NULL,
  subtotal DECIMAL(10,2) NOT NULL
);

INSERT INTO productos (...) VALUES (...);
```

Aplicando JDBC directo

- Funciona. Pero empieza a mezclar responsabilidades.



JDBC: leer productos

- El combo de la UI ahora carga productos desde la base.

```
public List<Producto> obtenerProductos() {
    List<Producto> productos = new ArrayList<>();
    String sql = "SELECT id, nombre, precio, emoji FROM productos";

    try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
         PreparedStatement statement = connection.prepareStatement(sql);
         ResultSet resultSet = statement.executeQuery()) {

        while (resultSet.next()) {
            productos.add(new Producto(
                resultSet.getInt("id"),
                resultSet.getString("nombre"),
                resultSet.getDouble("precio"),
                resultSet.getString("emoji")));
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error al obtener productos", e);
    }
    return productos;
}
```

JDBC: guardar pedido

- Usa transacción por si algo falla, no guarda el pedido incompleto.

```java
public int crearPedido(Pedido pedido, DescuentoStrategy descuento) throws SQLException {
    double subtotal = pedido.calcularSubtotal();
    double total = descuento.aplicarDescuento(subtotal);

    String insertPedidoSql = """
            INSERT INTO pedidos (subtotal, descuento_aplicado, total)
            VALUES (?, ?, ?)
            """;

    String insertItemSql = """
            INSERT INTO pedido_items (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
            VALUES (?, ?, ?, ?, ?)
            """;

    try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD)) {
        connection.setAutoCommit(false); // No guarda automáticamente para manejar transacciones, se debe hacer commit manualmente

        try (
            PreparedStatement pedidoStatement =
                    connection.prepareStatement(insertPedidoSql, Statement.RETURN_GENERATED_KEYS);

            PreparedStatement itemStatement =
                    connection.prepareStatement(insertItemSql)
        ) {
            pedidoStatement.setDouble(1, subtotal);
            pedidoStatement.setString(2, descuento.getNombre());
            pedidoStatement.setDouble(3, total);
            pedidoStatement.executeUpdate();

            int pedidoId = obtenerIdGenerado(pedidoStatement);

            for (ItemPedido item : pedido.getItems()) {
                Producto producto = item.getProducto();
                itemStatement.setInt(1, pedidoId);
                itemStatement.setInt(2, producto.getId());
                itemStatement.setInt(3, item.getCantidad());
                itemStatement.setDouble(4, producto.getPrecio());
                itemStatement.setDouble(5, item.calcularSubtotal());
                itemStatement.addBatch(); // Agrega el insert del item al batch para ejecutarlo después
            }

            itemStatement.executeBatch(); // Ejecuto todos los inserts de items en una sola operación para mejorar el rendimiento
            connection.commit(); // Si todo va bien, se confirma la transacción

            return pedidoId;

        } catch (SQLException e) {
            connection.rollback(); // Si ocurre un error, se revierte la transacción para mantener la integridad de los datos
            throw e;
        }
    }
}
```

Problemas

- SQL dentro del Service
  - PedidoService mezcla negocio + persistencia
- DriverManager repetido
  - Cada clase que accede a DB repite URL, user y password

Refactor: DAO + Singleton

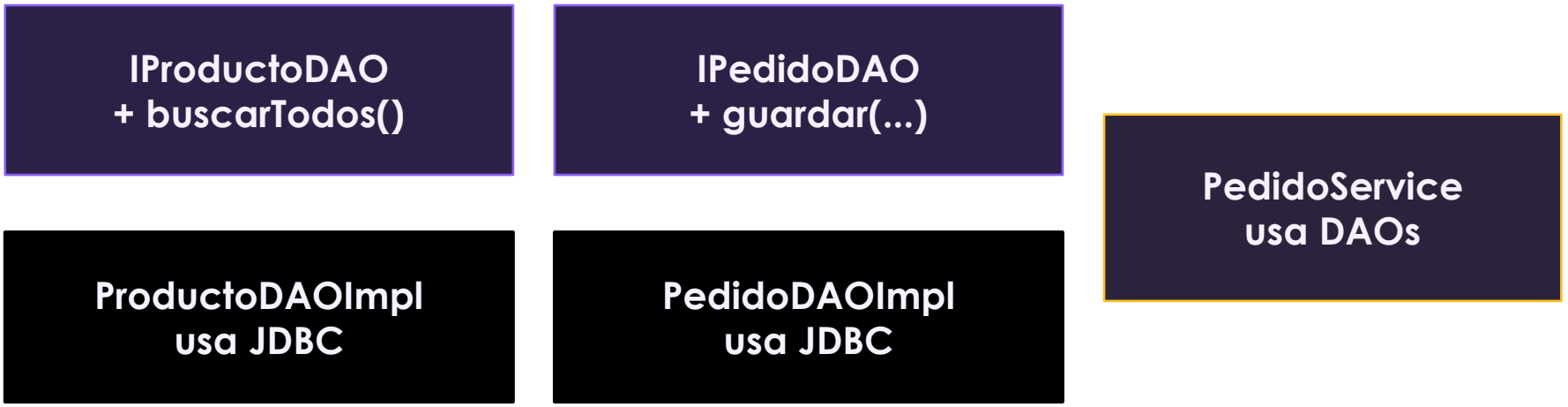
Patrón DAO



- **Indirection**: el Service no habla directo con MySQL.
- **Low Coupling**: el negocio depende de una interfaz DAO.
- **High Cohesion**: cada clase tiene una responsabilidad más clara.

Diseño con DAO

- DAO no es “la DB”: es el objeto que traduce entre objetos Java y tablas.



Patrón Singleton

- Una única instancia del gestor de conexiones.

DBConnection

private static DBConnection instance	private DBConnection()	public static getInstance()

- **NOTA**: en proyectos reales se suele usar un pool de conexiones. Acá lo usamos para enseñar el patrón.

Singleton: DBConnection

```java
public class DBConnection {

    private static DBConnection instance;

    private static final String URL =
        "jdbc:mysql://localhost:3306/magiccafe";
    private static final String USER = "root";
    private static final String PASSWORD = "root";

    private DBConnection() { }

    public static DBConnection getInstance() {
        if (instance == null) {
            instance = new DBConnection();
        }
        return instance;
    }

    public Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

DAO

```java
public interface PedidoDAO {
    int guardar(Pedido pedido, String nombreDescuento, double total);
}

public class PedidoDAOImpl implements PedidoDAO {
    @Override
    public int guardar(Pedido pedido, String nombreDescuento, double total) {
        try (Connection connection =
                DBConnection.getInstance().getConnection()) {

            connection.setAutoCommit(false);
            // INSERT INTO pedidos ...
            // INSERT INTO pedido_items ...
            connection.commit();
            return pedidoId;
        } catch (SQLException e) {
            throw new RuntimeException("Error al guardar", e);
        }
    }
}
```

Conclusión

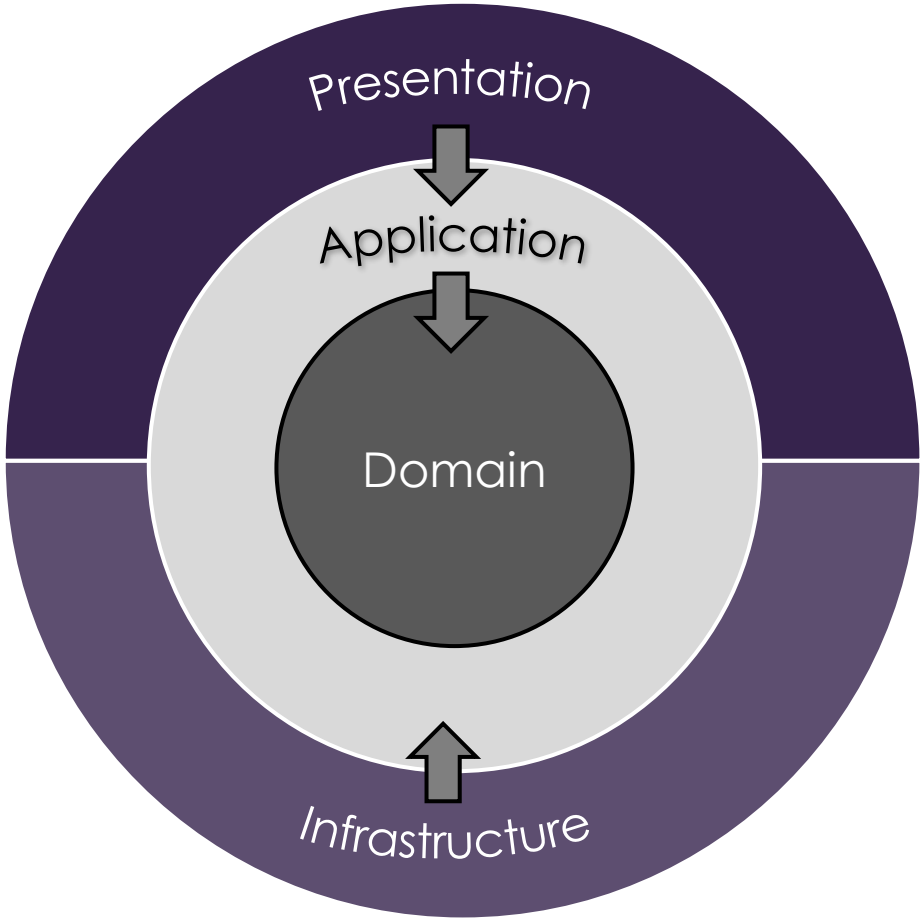
Aspecto	JDBC Directo	DAO + Singleton
Responsabilidad	Service calcula y también persiste.	Service calcula; DAO persiste.
Conexión	URL/user/password repetidos.	DBConnection centraliza.
Mantenibilidad	Crece desordenado.	Más fácil de extender.

Clean Architecture

- Es una forma de organizar el código separando responsabilidades.

"*La lógica de negocio debe estar separada de la tecnología.*"

Capas



- **Todas las flechas apuntan al Domain.**
  - Porque TODO depende del negocio.
  - Pero el negocio NO depende de nada.

Presentation

- La puerta de entrada
- Recibe requests/eventos
- Ejemplos:
  - Swing UI
  - REST Controller
  - React Frontend

Presentation

- Recibe HTTP
- Valida entrada básica
- Convierte DTOs
- Llama Services
- No debería tener negocio
- No debería tener SQL

```
@RestController
@RequestMapping("/api/pedidos")
public class PedidoController {
    @PostMapping
    public void crearPedido(@RequestBody PedidoDTO dto) {
        pedidoService.crearPedido(dto);
    }
}
```

Application

- Los casos de uso
- Acá viven los Services
- ¿Qué hace un service?
  - Coordina el flujo del sistema.
    - orquesta
    - coordina
    - ejecuta casos de uso
  - NO debería tener SQL

```
@Service
public class PedidoService {

    public void crearPedido(PedidoDTO dto) {
        ...
    }
}
```

Domain

- El corazón del negocio
- Acá viven:
  - entidades
  - reglas del negocio
  - comportamiento real

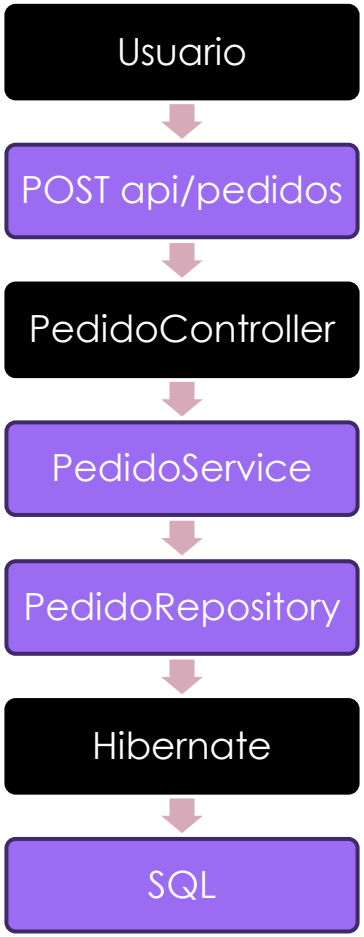
```
public class Pedido {

    public double calcularTotal() {
        ...
    }
}
```

Infrastructure

- La tecnología
- Acá viven:
  - MySQL
  - JPA
  - Hibernate
  - APIs externas
  - Email
  - Firebase
- IMPLEMENTA cosas y **NO** define negocio.

Flujo Completo – Crear pedido



Flujo: Crear pedido (Controller)

- SI ✅
  - Recibe HTTP
  - Recibe JSON
  - Convierte datos
  - Llama al Service
- NO ❌
  - SQL
  - Calcular totales
  - Lógica de negocio

```
@PostMapping
public void crearPedido(@RequestBody PedidoDTO dto) {

    pedidoService.crearPedido(dto);
}
```

Flujo: Crear pedido (Service)

- El Service coordina el caso de uso.
- Organiza el flujo
- Usa Repositories

```
public void crearPedido(PedidoDTO dto) {
    Producto producto =
        productoRepository.findById(dto.getProductoId());

    Pedido pedido = new Pedido();

    pedido.agregarItem(
        new ItemPedido(producto, dto.getCantidad())
    );

    pedidoRepository.save(pedido);
}
```

Flujo: Crear pedido (Entity)

- Aplica reglas de negocio:
  - cantidad > 0
  - calcularTotal
  - Etc.
- Protege invariantes

```java
public void agregarItem(ItemPedido item) {
    if(item.getCantidad() <= 0) {
        throw new IllegalArgumentException();
    }

    items.add(item);
}
```

¿Cuándo usar lógica en Service?

- Cuando la lógica:
  - involucra varias entidades
  - representa flujo/caso de uso
  - coordina repositorios
  - usa infraestructura

Ejemplo:

```
public void cerrarCaja() {
    // Lógica para cerrar la caja

}
```

- cerrarCaja() sería un método del **Service** porque coordina varias cosas:
  - pedidos
  - pagos
  - usuario/cajero
  - reporte
  - repositorios

CerrarCaja: Service

```java
public ReporteCajaDTO cerrarCaja(Long usuarioId) {

    List<Pedido> pedidosDelDia = pedidoRepository.buscarPedidosDelDia();
    List<Pago> pagosDelDia = pagoRepository.buscarPagosDelDia();

    Caja caja = new Caja(usuarioId);

    for (Pedido pedido : pedidosDelDia) {
        caja.agregarPedido(pedido);
    }

    for (Pago pago : pagosDelDia) {
        caja.agregarPago(pago);
    }

    caja.cerrar();

    cajaRepository.guardar(caja);

    return reporteCajaService.generarReporte(caja);
}
```

CerrarCaja: Entity

```java
public class Caja {

    private Long usuarioId;
    private List<Pedido> pedidos = new ArrayList<>();
    private List<Pago> pagos = new ArrayList<>();
    private boolean cerrada;
    private double totalFacturado;

    public Caja(Long usuarioId) {
        this.usuarioId = usuarioId;
    }

    public void agregarPedido(Pedido pedido) {
        if (cerrada) {
            throw new IllegalStateException("No se pueden agregar pedidos a una caja cerrada");
        }

        pedidos.add(pedido);
    }

    public void agregarPago(Pago pago) {
        if (cerrada) {
            throw new IllegalStateException("No se pueden agregar pagos a una caja cerrada");
        }

        pagos.add(pago);
    }

    public void cerrar() {
        if (cerrada) {
            throw new IllegalStateException("La caja ya está cerrada");
        }

        this.totalFacturado = pagos.stream()
                .mapToDouble(Pago::getMonto)
                .sum();

        this.cerrada = true;
    }
}
```

Conclusión

Clase	Conoce
Controller	HTTP/API
Service	Casos de uso
Entity	Negocio
Repository	DB

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

